PLAN DE PRUEBAS

SPRINT 2 — CARGA DE VENTAS Y TABLERO DE RENTABILIDAD

SIN-Quick

SQL Server · SSIS · Power BI

Dato	Detalle
Estudiante	Ferdinand Méndez
No. de carné	[0000-00-0000]
Curso	Seminario de Trabajo de Graduación
Empresa objeto de estudio	Óptica Quick, San Marcos, Guatemala
Versión del plan	1.0 — Sprint 2

Índice

1. Propósito **3**
2. Objetivos **3**
3. Alcance **3**
3.1 Incluido 3
3.2 Excluido 3
4. Tipos de prueba **4**
5. Estrategia de prueba **5**
5.1 Automatizar lo que se repite, observar lo que es humano 5
5.2 Principios 5
5.3 Adaptación de Selenium al proyecto 5
5.4 Criterios de entrada y salida 6
6. Parámetros del sistema **7**
6.1 Software 7
6.2 Hardware 7
6.3 Parámetros de negocio usados en las pruebas 7
7. Casos de prueba **9**
7.1 Unitaria 9
7.2 Integración 11
7.3 Sistema 12
7.4 Aceptación 13
7.5 Seguridad 13
7.6 Rendimiento 14
7.7 Humo (despliegue) 15
7.8 Manual (UX) 15
8. Ejecución del plan **17**
8.1 Dentro del pipeline 17
8.2 Reporte HTML de pytest 17
8.3 Registro del resultado obtenido 17
8.4 Resumen de resultados 17
9. Gestión de defectos **18**
10. Riesgos del plan **18**

1. Propósito

Este plan define cómo se verifica que la solución de inteligencia de negocios de Óptica Quick hace lo que promete antes de que el propietario tome una decisión con sus cifras. Cubre lo construido en el Sprint 2: la base de datos en SQL Server, la carga de ventas desde el punto de venta mediante SSIS y el tablero de ventas y rentabilidad en Power BI.

El plan parte de una idea sencilla: en un sistema de este tipo el error más caro no es que algo se caiga, sino que **muestre una cifra equivocada y nadie lo note**. Por eso buena parte de las pruebas no verifica que el sistema funcione, sino que los números coincidan con la realidad del negocio.

2. Objetivos

1. Verificar que las reglas de negocio RN-01, RN-03, RN-04, RN-05, RN-09 y RN-10 se calculan como fueron especificadas.
2. Comprobar que la carga de ventas es confiable: aparta los registros con error y su motivo, no duplica facturas y no deja datos a medias cuando falla.
3. Demostrar que el total del tablero coincide con lo facturado en el punto de venta (criterio de aceptación de la historia de conciliación).
4. Confirmar que la cuenta que usa Power BI no puede leer datos sensibles ni ejecutar la carga.
5. Medir los tiempos de carga y consulta contra los requerimientos de rendimiento RR-01, RR-02, RR-04 y RR-05.
6. Ejecutar todas las pruebas automatizadas en cada cambio, dentro del pipeline de Jenkins, y dejar un reporte revisable de cada ejecución.

3. Alcance

3.1 Incluido

Componente	Qué se prueba
Base de datos SQL Server	Tablas, restricciones, esquemas, roles y permisos del proyecto SINQuick.DB
Procedimientos de carga (esquema etl)	Carga del archivo, validación, umbral de rechazo, carga al modelo dimensional
Vistas para Power BI (esquema rpt)	Cálculos de ingreso, margen, alertas, ciclo de renovación y conciliación
Paquete SSIS CargaVentas.dtsx	Despliegue en el catálogo SSISDB y ejecución de verificación en staging
Despliegue	Que staging quede completo después de cada entrega (prueba de humo)
Tablero de Power BI	Tiempos de apertura y de filtros, y comprensión por parte del propietario (pruebas manuales)

3.2 Excluido

- **Inventario, clientes y campañas**: se prueban en los sprints 3 a 5, cuando se construyan.

- **El punto de venta**: es un sistema existente; solo se prueba lo que SIN-Quick hace con su exportación.
- **Pruebas automatizadas de interfaz con Selenium**: ver la sección 5.3.

4. Tipos de prueba

Tipo	Qué verifica	Herramienta	Casos
Unitarias	Cada regla de negocio de forma aislada, con tablas falsas	tSQLt	CP-01 a CP-09
Integración	Que archivo, procedimientos, modelo dimensional y vistas funcionen juntos	pytest + pyodbc	CP-10 a CP-12
Sistema	Comportamiento completo de la carga: repetición y fallas	pytest + pyodbc	CP-13 a CP-14
Aceptación	Criterios de aceptación de las historias de usuario del sprint	pytest + pyodbc	CP-15 a CP-17
Seguridad	Permisos por rol e inyección SQL	pytest + pyodbc	CP-18 a CP-20
Rendimiento	Tiempo de carga del histórico y de consulta del tablero	pytest + pyodbc	CP-21 a CP-22
Humo	Que el despliegue en staging quedó completo	pytest + pyodbc	CP-23
Manuales (UX)	Tiempos percibidos y comprensión del tablero	Observación y cronómetro	CP-24 a CP-27

Total: 27 casos de prueba, de los cuales 23 son automatizados y 4 manuales.

5. Estrategia de prueba

5.1 Automatizar lo que se repite, observar lo que es humano

Las pruebas se organizan en tres niveles. La mayoría son **unitarias en tSQLt**: prueban cada regla con tablas falsas, corren en segundos y señalan exactamente qué regla se rompió. Encima están las **pruebas de pytest**, que ejecutan la carga real con archivos CSV y consultan el resultado, porque una regla puede estar bien escrita y aun así fallar al combinarse con las demás. Y en la cima están las **pruebas manuales**, reservadas para lo que ninguna herramienta puede juzgar: si el propietario entiende lo que ve.

Nivel	Cuántas	Cuándo corren	Qué detectan
Unitarias (tSQLt)	9	En cada commit, en la etapa Test	Una regla de negocio mal calculada
pytest	14	En cada commit; las de humo, en cada despliegue	Fallas de integración, datos a medias, permisos, lentitud
Manuales	4	En la Sprint Review, con el propietario	Un tablero correcto que nadie entiende

5.2 Principios

- **Aislamiento.** Las pruebas destructivas solo corren en la base `SINQuick_CI`, que el pipeline recrea desde cero en cada ejecución. El código de las pruebas se niega a limpiar cualquier base que no termine en `_CI`.
- **Datos controlados.** Cada prueba crea los archivos y el catálogo que necesita. No se usan datos reales de clientes: las graduaciones son información de salud.
- **Una prueba, un caso.** Cada prueba lleva el ID de su caso en el nombre o en una marca, de modo que el reporte se lee contra este plan sin traducción.
- **Las pruebas deben poder fallar.** Cada caso tiene un umbral o un valor exacto. Si una prueba no puede fallar en ningún escenario, no está probando nada.
- **Nada llega a staging sin pasar las pruebas.** Si la etapa Test falla, el pipeline no despliega.

5.3 Adaptación de Selenium al proyecto

El lineamiento del curso propone automatizar la interfaz con Selenium. En esta solución la interfaz es **Power BI**, y ahí Selenium no se adapta bien por dos razones:

- El informe publicado exige iniciar sesión con una cuenta de Microsoft, normalmente con autenticación multifactor. Automatizar ese inicio de sesión obliga a guardar credenciales en el pipeline o a desactivar protecciones, lo cual contradice el requerimiento de seguridad RNF-06.
- Power BI dibuja sus visuales en un lienzo: no expone los valores como texto en la página, por lo que Selenium solo podría comparar capturas, que se rompen con cualquier cambio de diseño.

La decisión fue **probar los datos donde se calculan**: las vistas del esquema `rpt` que alimentan el tablero se verifican con tSQLt y pytest. Si esas vistas dan la cifra correcta, el tablero la muestra. Lo que Power BI sí agrega —tiempos de respuesta y claridad para el usuario— se cubre con las pruebas manuales CP-24 a CP-27.

5.4 Criterios de entrada y salida

Criterio	Condición
Entrada	El DACPAC y el paquete SSIS compilan (etapa Build en verde) y la base de pruebas se desplegó.
Salida del sprint	100 % de las pruebas unitarias, de aceptación y de seguridad aprobadas.
Salida del sprint	Pruebas de rendimiento dentro de sus umbrales.
Salida del sprint	Pruebas manuales ejecutadas con el propietario y registradas.
Suspensión	Si la base de pruebas no se puede desplegar, se detiene la ejecución: no se prueba sobre una base incompleta.

6. Parámetros del sistema

Las pruebas se ejecutan en el agente de Jenkins. El reporte HTML de pytest registra automáticamente el sistema operativo, la versión de Python, el procesador, el número de ejecución y el commit probado, de modo que cada reporte queda ligado a su entorno. La columna *Versión verificada* se completa con los valores de ese reporte.

6.1 Software

Componente	Versión objetivo	Versión verificada
Sistema operativo del agente	Windows 10/11 de 64 bits	[del reporte]
Motor de base de datos	SQL Server 2022 Developer (CI y staging)	[SELECT @@VERSION]
ETL	SQL Server Integration Services 2022 + catálogo SSISDB	[completar]
Diseño del ETL	Visual Studio 2022 + SSIS Projects	[completar]
Informe	Power BI Desktop (versión vigente)	[completar]
Navegador para Power BI Service	Microsoft Edge o Google Chrome, versión vigente	[completar]
Aplicación móvil	Power BI Mobile (Android o iOS)	[completar]
Pruebas unitarias	tSQLt (versión vigente)	[completar]
Pruebas de integración	Python 3.12, pytest 8 o superior, pytest-html 4, pyodbc 5	[del reporte]
Controlador	ODBC Driver 18 for SQL Server	[completar]
Integración continua	Jenkins LTS, .NET SDK 8, SqlPackage, SSIS DevOps Tools	[completar]

6.2 Hardware

Equipo	Especificación
Agente de Jenkins (pruebas)	Procesador de 4 núcleos, 16 GB de RAM, SSD (equipo del estudiante)
Servidor del local (producción)	Procesador de 4 núcleos, 16 GB de RAM, SSD de 480 GB, UPS de 650 VA (RI-05, RI-06)
Consulta móvil	Teléfono del propietario con conexión 4G

6.3 Parámetros de negocio usados en las pruebas

Parámetro	Valor	Dónde se configura
RN-01 Ciclo de renovación	18 meses	dbo.ParametroRegla
RN-04 Inventario sin movimiento	180 días	dbo.ParametroRegla
UMB-RECH Rechazo máximo por carga	5 %	dbo.ParametroRegla

Parámetro	Valor	Dónde se configura
Carpeta de entrada de pruebas	C:\SINQuick\CI\entrada	Variable ETL_CARPETA del Jenkinsfile
Base de pruebas / staging	SINQuick_CI / SINQuick_Staging	Variables BD_CI y BD_STAGING del Jenkinsfile

7. Casos de prueba

Cada caso tiene su ID, precondiciones, pasos y resultado esperado. Las columnas *Resultado obtenido* y *Estado* se llenan con la ejecución real: el script `docs/plan-pruebas/completar_resultados.py` las toma de los reportes del pipeline (sección 8.3).

7.1 Unitaria

ID	CP-01
Nombre	**RN-01 marca el ciclo vencido a los 18 meses**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Tablas Cliente, Venta y ParametroRegla aisladas con FakeTable; RN-01 = 18.
Pasos	1. Registrar un cliente con compra hace 20 meses y otro hace 3 meses. 2. Consultar rpt.vw_ClientesRenovacion.
Resultado esperado	Cliente de 20 meses: cicloVencido = 1. Cliente de 3 meses: cicloVencido = 0.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-02
Nombre	**RN-01 usa el parámetro configurado y no un valor fijo**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	RN-01 = 12 en ParametroRegla.
Pasos	1. Registrar un cliente con compra hace 14 meses. 2. Consultar la vista.
Resultado esperado	cicloVencido = 1: la vista obedece al parámetro.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-03
Nombre	**RN-09 solo es contactable quien tiene teléfono, consentimiento y no pidió exclusión**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Cuatro clientes con ciclo vencido y distinta condición de contacto.
Pasos	1. Consultar contactable y motivoExclusion en la vista.
Resultado esperado	Solo el cliente completo es contactable; los otros tres muestran SIN CONSENTIMIENTO, SIN TELEFONO y EXCLUSION SOLICITADA.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-04
Nombre	**RN-03 detecta la existencia bajo el nivel mínimo**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Dos productos: uno con 2 unidades y mínimo 5, otro con 14 y mínimo 4.
Pasos	1. Consultar rpt.vw_InventarioAlertas.
Resultado esperado	ARM-RB3025 bajoMinimo = 1; ARM-VG1180 bajoMinimo = 0.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-05
Nombre	**RN-04 marca el inventario sin movimiento y calcula su valor**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	RN-04 = 180 días; un producto sin movimiento hace 200 días y otro hace 5.
Pasos	1. Consultar rpt.vw_InventarioAlertas.
Resultado esperado	El de 200 días queda sin movimiento con valor Q 2,590.00; el de 5 días no.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-06
Nombre	**El margen excluye ventas anuladas y reposiciones en garantía**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Una línea normal, una en garantía y una de venta anulada.
Pasos	1. Consultar rpt.vw_VentasDetalle.
Resultado esperado	Solo aparece la línea normal: ingreso Q 680, costo Q 360, margen Q 320.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-07
Nombre	**Una cantidad inválida se aparta con su motivo**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Dos líneas en staging, una con cantidad 'dos'.
Pasos	1. Ejecutar etl.usp_ValidarVentas con umbral 100 %.
Resultado esperado	Un registro en RegistroRechazado con motivo 'Cantidad inválida'.
Resultado obtenido	*Pendiente de ejecución*

Estado	*Pendiente*

ID	**CP-08**
Nombre	**La carga se detiene si el rechazo supera el umbral**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Dos líneas en staging, una con producto inexistente (50 % de rechazo).
Pasos	1. Ejecutar etl.usp_ValidarVentas con umbral 5 %.
Resultado esperado	El procedimiento lanza el error 50020.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-09**
Nombre	**Una factura ya cargada se omite sin contarse como rechazo**
Tipo / herramienta	Unitaria · tSQLt
Precondiciones	Factura F-1 ya existe en dbo.Venta y vuelve a llegar en el archivo.
Pasos	1. Ejecutar etl.usp_ValidarVentas con umbral 0 %.
Resultado esperado	La línea queda OMITIDA y la carga registra 0 rechazos.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.2 Integración

ID	**CP-10**
Nombre	**Carga completa de un archivo válido del punto de venta**
Tipo / herramienta	Integración · pytest
Precondiciones	Base de pruebas desplegada desde el DACPAC; catálogo de productos y asesores cargado.
Pasos	1. Colocar un CSV válido de 3 facturas y 4 líneas en la carpeta de entrada. 2. Ejecutar etl.usp_EjecutarCargaVentas.
Resultado esperado	Resultado EXITOSA; 4 líneas procesadas e incorporadas; 3 ventas en dbo.Venta.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-11**
Nombre	**Una carga con errores aparta los rechazados y carga el resto**
Tipo / herramienta	Integración · pytest

Precondiciones	Umbral de rechazo configurado en 50 %.
Pasos	1. Cargar un CSV con 3 líneas válidas y 1 con producto inexistente.
Resultado esperado	Resultado PARCIAL; 3 incorporadas, 1 rechazada con su motivo en rpt.vw_RechazosPorCarga.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-12**
Nombre	**Las vistas de Power BI exponen los datos cargados**
Tipo / herramienta	Integración · pytest
Precondiciones	Carga de CP-10 realizada.
Pasos	1. Consultar rpt.vw_VentasDetalle y sumar el ingreso.
Resultado esperado	Las líneas y el ingreso coinciden con el archivo, sin anuladas ni garantías.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.3 Sistema

ID	**CP-13**
Nombre	**Una carga repetida no duplica ventas (idempotencia)**
Tipo / herramienta	Sistema · pytest
Precondiciones	Archivo ya cargado una vez.
Pasos	1. Ejecutar la misma carga por segunda vez.
Resultado esperado	Resultado EXITOSA, 0 líneas incorporadas y el número de ventas no cambia.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-14**
Nombre	**Una carga fallida no deja datos a medias**
Tipo / herramienta	Sistema · pytest
Precondiciones	Umbral de rechazo en 5 %.
Pasos	1. Cargar un CSV con 50 % de líneas inválidas.
Resultado esperado	Error 50020; carga FALLIDA en la bitácora; dbo.Venta sin cambios; rechazos registrados.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.4 Aceptación

ID	CP-15
Nombre	**El total del mes coincide con lo facturado en el punto de venta**
Tipo / herramienta	Aceptación · pytest
Precondiciones	Archivo de un mes con ventas normales y una anulada.
Pasos	1. Cargar el archivo. 2. Comparar rpt.vw_ConciliacionMensual contra el total del archivo sin anuladas.
Resultado esperado	Diferencia de Q 0.00 (criterio de aceptación de US-14).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-16
Nombre	**Un cliente nuevo aparece por renovar pero no es contactable sin consentimiento**
Tipo / herramienta	Aceptación · pytest
Precondiciones	Venta de hace 20 meses a un cliente que no existía.
Pasos	1. Cargar el archivo. 2. Consultar rpt.vw_ClientesRenovacion.
Resultado esperado	cicloVencido = 1, contactable = 0, motivo SIN CONSENTIMIENTO (RN-01 y RN-09).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	CP-17
Nombre	**La carga nocturna procesa el volumen de un día en el tiempo exigido**
Tipo / herramienta	Aceptación · pytest
Precondiciones	Archivo con 300 líneas (el doble de un día de alta venta).
Pasos	1. Ejecutar la carga y medir el tiempo.
Resultado esperado	Termina en menos de 10 minutos (RR-04).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.5 Seguridad

ID	CP-18
Nombre	**La cuenta de Power BI no puede leer graduaciones ni tablas base**

Tipo / herramienta	Seguridad · pytest
Precondiciones	Usuario de prueba miembro de rol_reportes.
Pasos	1. Como ese usuario, consultar dbo.Graduacion y dbo.Venta. 2. Consultar rpt.vw_VentasDetalle.
Resultado esperado	1: permiso denegado (error 229). 2: la consulta funciona.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-19**
Nombre	**Una ruta de archivo con inyección SQL es rechazada**
Tipo / herramienta	Seguridad · pytest
Precondiciones	Base de pruebas desplegada.
Pasos	1. Ejecutar la carga con la ruta "x'; DROP TABLE dbo.Venta; --".
Resultado esperado	Error 50010 y dbo.Venta sigue existiendo.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-20**
Nombre	**La cuenta de Power BI no puede ejecutar la carga**
Tipo / herramienta	Seguridad · pytest
Precondiciones	Usuario de prueba miembro de rol_reportes.
Pasos	1. Como ese usuario, ejecutar etl.usp_EjecutarCargaVentas.
Resultado esperado	Permiso denegado (error 229).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.6 Rendimiento

ID	**CP-21**
Nombre	**Carga inicial del histórico**
Tipo / herramienta	Rendimiento · pytest
Precondiciones	Archivo con 25,000 líneas (volumen histórico estimado de la óptica).
Pasos	1. Ejecutar la carga INICIAL y medir el tiempo.
Resultado esperado	Termina en menos de 4 horas (RR-05); se espera que sea de minutos.
Resultado obtenido	*Pendiente de ejecución*

Estado	*Pendiente*

ID	**CP-22**
Nombre	**Consulta de ventas del mes para el tablero**
Tipo / herramienta	Rendimiento · pytest
Precondiciones	Histórico de CP-21 cargado.
Pasos	1. Consultar rpt.vw_VentasDetalle filtrado por un mes, 10 veces.
Resultado esperado	Cada consulta tarda menos de 2 segundos (margen dentro de RR-02).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.7 Humo (despliegue)

ID	**CP-23**
Nombre	**El despliegue en staging quedó completo**
Tipo / herramienta	Humo (despliegue) · pytest
Precondiciones	Pipeline en la etapa Deploy Staging.
Pasos	1. Verificar esquemas, vistas, procedimientos, roles, parámetros y calendario en SINQuick_Staging.
Resultado esperado	Todos los objetos existen; RN-01 = 18; DimTiempo cubre 2019-2030.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

7.8 Manual (UX)

ID	**CP-24**
Nombre	**El tablero de ventas abre en el tiempo exigido**
Tipo / herramienta	Manual (UX) · Manual
Precondiciones	Tablero publicado; equipo del local.
Pasos	1. Abrir el tablero de ventas y rentabilidad y cronometrar.
Resultado esperado	Abre en 5 segundos o menos (RR-01).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-25**

Nombre	**Los filtros responden en el tiempo exigido**
Tipo / herramienta	Manual (UX) · Manual
Precondiciones	Tablero abierto.
Pasos	1. Filtrar por asesor, por proveedor y por mes, cronometrando cada uno.
Resultado esperado	Cada filtro responde en 3 segundos o menos (RR-02).
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-26**
Nombre	**El propietario interpreta el tablero sin ayuda**
Tipo / herramienta	Manual (UX) · Manual
Precondiciones	Propietario frente al tablero, sin explicación previa.
Pasos	1. Preguntarle qué categoría deja más margen y qué asesor vendió más el mes pasado.
Resultado esperado	Responde correctamente leyendo solo el tablero.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

ID	**CP-27**
Nombre	**Consulta desde el teléfono del propietario**
Tipo / herramienta	Manual (UX) · Manual
Precondiciones	Power BI Mobile instalado con la cuenta del propietario.
Pasos	1. Abrir el tablero en el teléfono. 2. Revisar ventas del mes y margen.
Resultado esperado	Todo es legible en la vista móvil sin desplazamiento horizontal.
Resultado obtenido	*Pendiente de ejecución*
Estado	*Pendiente*

8. Ejecución del plan

8.1 Dentro del pipeline

El plan se ejecuta completo en cada cambio que llega al repositorio. La etapa **Test** del Jenkinsfile recrea la base `SINQuick_CI`, instala tSQLt, corre las 9 pruebas unitarias y luego las 13 de pytest. La etapa **Deploy Staging** corre la prueba de humo contra `SINQuick_Staging` después de desplegar.

Paso	Script	Salida
Desplegar base de pruebas	ci\publicar-db.ps1 -Base SINQuick_CI -Recrear	Base recreada desde el DACPAC
Pruebas unitarias	ci\pruebas-tsqlt.ps1	reportes\tsqlt.xml (JUnit)
Pruebas pytest	ci\pruebas-pytest.ps1	reportes\pytest.html, pytest.xml y resultados-pytest.json
Prueba de humo	ci\pruebas-pytest.ps1 -Humo	reportes\pytest-staging.html y .xml

8.2 Reporte HTML de pytest

pytest genera `reportes/pytest.html` como archivo autocontenido. Jenkins lo publica en cada ejecución (enlace *Reporte de pruebas (pytest)*). El reporte incluye:

- El encabezado con el entorno: sistema operativo, Python, procesador, número de ejecución, commit y base de datos probada.
- El resumen de aprobadas, fallidas y omitidas, con la duración total.
- Una fila por prueba con las columnas **Caso** y **Tipo**, agregadas para que cada fila se lea directamente contra este plan.
- El detalle del error de cada prueba fallida, con la aserción que no se cumplió.

Los resultados de tSQLt aparecen en la vista **Test Result** de Jenkins junto con los de pytest, porque ambos se publican en formato JUnit.

8.3 Registro del resultado obtenido

Después de una ejecución, con los archivos de `reportes/` descargados de Jenkins:

```
python docs\plan-pruebas\completar_resultados.py
```

El script lee `tsqlt.xml` y `resultados-pytest.json`, busca cada caso por su ID en este documento y escribe el resultado obtenido y el estado (Aprobado o Fallido). Guarda una copia con el sufijo `_ejecutado`. Los casos manuales se completan a mano durante la Sprint Review.

8.4 Resumen de resultados

Tipo	Casos	Aprobados	Fallidos	Pendientes
Unitaria	9			
Integración	3			

Tipo	Casos	Aprobados	Fallidos	Pendientes
Sistema	2			
Aceptación	3			
Seguridad	3			
Rendimiento	2			
Humo (despliegue)	1			
Manual (UX)	4			
Total	**27**			

9. Gestión de defectos

Severidad	Criterio	Tratamiento
Crítica	Una cifra del tablero es incorrecta o se expone un dato sensible	Bloquea el despliegue; se corrige en el mismo sprint
Alta	La carga falla o deja datos a medias	Se corrige antes de la Sprint Review
Media	Un tiempo de respuesta supera su umbral	Se registra en el backlog con prioridad
Baja	Un texto o un formato del tablero confunde	Se registra en el backlog

Cada defecto se registra como *Bug* en Azure Boards, vinculado a la historia de usuario afectada y al caso de prueba que lo detectó.

10. Riesgos del plan

Riesgo	Mitigación
El formato real de la exportación del punto de venta difiere del supuesto	La historia US-10 lo valida el primer día; las pruebas usan el mismo formato que se documente
Las pruebas pasan con datos sintéticos pero fallan con datos reales	La prueba de rendimiento usa un volumen de 25,000 líneas similar al histórico real
Las pruebas manuales dependen de la disponibilidad del propietario	Se agendan dentro de la Sprint Review, que ya cuenta con su presencia